

DSA Q&A — Linked List in C

1. Why the error: unknown type name 'Node'?

Error:

```
linkedList01.c:6:5: error: unknown type name 'Node'
```

Reason:

In **C**, defining `struct Node { ... };` creates a type named `struct Node` — not `Node` alone. The bare name `Node` means nothing to the compiler. This is unlike C++, where `struct Node` automatically makes `Node` usable as a plain type.

Fix 1 — Use `struct` everywhere:

```
struct Node
{
    int data;
    struct Node * next;
};

int main() {
    struct Node * head;
}
```

Fix 2 — Use `typedef` (alias):

```
typedef struct Node
{
    int data;
    struct Node * next; // alias not complete yet, still needs struct
} Node;

int main() {
    Node * head; // now Node alone works
}
```

Inside the struct body, the alias `Node` isn't defined yet, so the self-referential pointer must still say `struct Node *`.

2. How to create a new node with malloc?

```
struct Node *newNode = (struct Node *) malloc(sizeof(struct Node));
```

Part	Meaning
<code>malloc(sizeof(struct Node))</code>	allocates exactly enough bytes for one Node
<code>(struct Node *)</code>	casts the returned <code>void *</code> to the right pointer type
<code>newNode</code>	points to that allocated memory

Then set its fields:

```
newNode->data = data;
newNode->next = NULL;
```

Always include `<stdlib.h>` at the top for `malloc`.

3. Why is the list not creating when `head` is uninitialized?

Problem code:

```
struct Node *head;
head->data = 700; // ❌ head is uninitialized – points to garbage
head->next = NULL;
```

Reason:

`head` is just a pointer with no memory behind it. It contains a random garbage address. So `list == NULL` is **false** (garbage $\neq$ NULL), the `if` block is skipped, and `createList` returns `NULL` without creating anything.

Fix:

```
struct Node *head = NULL; // ✅ explicitly initialize to NULL
head = createList(head, 700);
```

	<code>list == NULL?</code>	Result
<code>head</code> uninitialized	No (random garbage)	skips <code>if</code> , returns <code>NULL</code>
<code>head = NULL</code>	Yes	creates node, returns it

Rule: In C, local pointers are **never automatically initialized to NULL** — you must do it explicitly.

4. Can we avoid returning from `createList` using pointer to pointer?

Yes! By passing `struct Node **` (pointer to pointer), the function can modify `head` directly — no return needed.

Old way (`struct Node *`):

```
// must return and reassign
head = createList(head, i);
```

New way (`struct Node **`):

```
void createList(struct Node **list, int data) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;

    if (*list == NULL) {
        *list = newNode;
        return;
    }

    struct Node *temp = *list;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}

// in main:
createList(&head, i);    // pass address of head
```

	What's passed	Can modify original <code>head</code> ?
<code>struct Node *</code>	copy of the pointer	❌ No
<code>struct Node **</code>	address of the pointer	✅ Yes

- `*list` → the actual `head` pointer
- `**list` → the data inside the node `head` points to

5. Is the new code (pointer to pointer) more efficient?

No — equally efficient in performance.

	Old code	New code
Traversal to last node	$O(n)$	$O(n)$
Node creation (malloc)	$O(1)$	$O(1)$

	Old code	New code
Memory used	same	same

The improvement is purely **code design / style**, not efficiency:

-  Cleaner **main**
-  Clearer responsibility of the function
-  No speed gain
-  No memory gain

6. Tail pointer — $O(1)$ insert instead of $O(n)$

Problem with current approach:

Every insert walks the whole list to find the last node → **$O(n)$ per insert**.

Solution — Keep a **tail pointer**:

tail always points to the last node, so insert is **$O(1)$** directly.

```
head → [1] → [2] → [3] → NULL
                ↑
                tail
```

Code:

```
void createList(struct Node **head, struct Node **tail, int data) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
        *tail = newNode;
        return;
    }

    (*tail)->next = newNode; // link new node at the end
    *tail = newNode;        // move tail forward
}

// in main:
struct Node *head = NULL;
struct Node *tail = NULL;

for (i = 1; i < 100; i++) {
    createList(&head, &tail, i);
}
```

	Without tail	With tail pointer
Per insert	$O(n)$ — walks full list	$O(1)$ — direct jump
99 inserts total	~4950 steps	99 steps
Extra memory	none	just one pointer

7. Are these two statements the same?

```
// Option 1
(*tail)->next = newNode;
*tail = newNode;

// Option 2
(*tail)->next = newNode;
*tail = (*tail)->next;
```

In this specific case — yes, same result.

After the first line, `(*tail)->next` is already `newNode`, so both options end up with `*tail` pointing to `newNode`.

But Option 1 is better because:

- Clear and direct — *"tail is now the new node"*
- No dependency on the previous line's side effect
- Option 2 is fragile — if order changes, it silently breaks
- Option 1 is self-explanatory; Option 2 requires more thinking

Rule: Always prefer **explicit** over **clever**.